

Experiment – 2

Student Name: Mimanshu Gahlaut

Branch: CSE-AIML

Semester: 5th

Subject Name: ADBMS

UID: 24BAI70038

Section/Group: 24AIT_KRG-1_B

Date of Performance: 13/7/26

Subject Code: 24CSP-310

(A)

A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments. The solution must join the student profile, course subjects, and grade marks tables, returning a breakdown sorted by the highest average marks down to the lowest.

Output

Department	Average Marks
Mathematics	95.00
Computer Science	83.33

Solution:

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50) NOT NULL,  
    dept_id INT REFERENCES Departments(dept_id)  
);
```

```
CREATE TABLE Subjects (  
    subject_id INT PRIMARY KEY,  
    subject_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Marks (  
    student_id INT REFERENCES Students(student_id),  
    subject_id INT REFERENCES Subjects(subject_id),  
    marks_scored INT NOT NULL,  
    PRIMARY KEY (student_id, subject_id)  
);
```

```

INSERT INTO Departments (dept_id, dept_name) VALUES
(1, 'Computer Science'),
(2, 'Mathematics');

INSERT INTO Students (student_id, student_name, dept_id) VALUES
(101, 'Alex', 1),
(102, 'Bella', 1),
(103, 'Charlie', 2);

INSERT INTO Subjects (subject_id, subject_name) VALUES
(501, 'Data Structures'),
(502, 'Calculus');

INSERT INTO Marks (student_id, subject_id, marks_scored) VALUES
(101, 501, 85),
(101, 502, 90),
(102, 501, 75),
(103, 502, 95);

SELECT D.DEPT_NAME AS DEPARTMENT, ROUND (AVG (M.MARKS_SCORED) , 2) AS AVERAGE_MARKS
FROM DEPARTMENTS AS D
JOIN STUDENTS AS S
ON S.DEPT_ID =D.DEPT_ID
JOIN MARKS AS M
ON M.STUDENT_ID=S.STUDENT_ID
GROUP BY D.DEPT_NAME
ORDER BY AVERAGE_MARKS DESC;

```

(B)

Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to **merge these datasets** and identify **each unique employee** (by EmpID) along with their **lowest recorded salary** across both systems.

Objective

1. Combine two tables A and B.
2. Return each EmpID with their **lowest salary**, and the corresponding **Ename**.

Table A

EmpID	Ename	Salary
1	AA	1000
2	BB	300

Table B

EmpID	Ename	Salary
2	BB	400
3	CC	100

Output

EmpID	Ename	Salary
1	AA	1000
2	BB	300
3	CC	100

Solution:

```
CREATE TABLE A
```

```
(  
    EmpID INT,  
    Ename VARCHAR(50),  
    Salary INT  
);
```

```
CREATE TABLE B
```

```
(  
    EmpID INT,  
    Ename VARCHAR(50),  
    Salary INT  
);
```

```
INSERT INTO A (EmpID, Ename, Salary)  
VALUES  
(1, 'AA', 1000),  
(2, 'BB', 300);
```

```
INSERT INTO B (EmpID, Ename, Salary)  
VALUES  
(2, 'BB', 400),  
(3, 'CC', 100);
```

```
SELECT EMPID, MIN(ENAME) AS ENAME, MIN(SALARY) AS SALARY  
FROM(  
    SELECT * FROM A  
    UNION ALL  
    SELECT * FROM B  
)  
GROUP BY EMPID  
ORDER BY EMPID;
```

(C)

A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report.

If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the **Department Name**, **Employee Name**, and **Salary**, arranged in ascending order of department name.

Note: The employee and department information are maintained in separate normalized tables.

Input Table: Employee				Department	
ID	NAME	SALARY	DEPT_ID	ID	DEPT_NAME
1	JOE	70000	1	1	IT
2	JIM	90000	1	2	SALES
3	HENRY	80000	2		
4	SAM	60000	2		
4	MAX	90000	1		

OUTPUT		
DEPT_NAME	NAME	SALARY
IT	MAX	90000
IT	JIM	90000
SALES	HENRY	80000

Solution:

(a)

```
CREATE TABLE Department
(
    id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

CREATE TABLE Employee
(
    id INT PRIMARY KEY,
    name VARCHAR(50),
    salary INT,
    department_id INT,
    FOREIGN KEY(department_id)
    REFERENCES Department(id)
);

INSERT INTO Department
VALUES
(1, 'IT'),
(2, 'SALES');
```

```
INSERT INTO Employee
VALUES
(1, 'JOE', 70000, 1),
(2, 'JIM', 90000, 1),
(3, 'HENRY', 80000, 2),
(4, 'SAM', 60000, 2),
(5, 'MAX', 90000, 1);
```

```
SELECT D.DEPT_NAME, E.NAME, E.SALARY
FROM EMPLOYEE AS E
JOIN DEPARTMENT AS D
ON E.DEPARTMENT_ID = D.ID
WHERE E.SALARY IN(
    SELECT MAX(SALARY)
    FROM EMPLOYEE AS E1
    WHERE E1.DEPARTMENT_ID = E.DEPARTMENT_ID
)
ORDER BY DEPT_NAME;
```

(b)

```
SELECT D.DEPT_NAME, E.NAME, E.SALARY
FROM EMPLOYEE AS E
JOIN DEPARTMENT AS D
ON E.DEPARTMENT_ID = D.ID
WHERE E.SALARY IN(
    SELECT MAX(SALARY)
    FROM EMPLOYEE AS E1
    WHERE E1.DEPARTMENT_ID = E.DEPARTMENT_ID
    AND E1.SALARY NOT IN(
        SELECT MAX(SALARY)
        FROM EMPLOYEE AS E2
        WHERE E2.DEPARTMENT_ID = E1.DEPARTMENT_ID
    )
)
ORDER BY DEPT_NAME;
```